

Guía personal para el demo

Jupyter Notebooks en VS Code con Codex

Propósito: recuperar rápidamente el flujo de trabajo de Jupyter dentro de Visual Studio Code y ejecutar con seguridad el demo de pérdida silenciosa de observaciones.

Diseñado para: Kristian López Vargas Entorno principal: macOS, Visual Studio Code, Python, Jupyter y la extensión oficial de Codex Actualización: agosto de 2026

1. La idea del entorno

Durante el demo, VS Code será el laboratorio visible. En una sola ventana tendrás:

- el explorador de archivos a la izquierda;
- el notebook en el centro;
- Codex en un panel lateral a la derecha;
- la salida de cada celda debajo del código;
- la terminal integrada disponible, pero cerrada mientras no se necesite.

No necesitas iniciar un servidor Jupyter manualmente. La extensión de Jupyter puede ejecutar el notebook usando un entorno local de Python que contenga `ipykernel`.

El principio operativo es:

Codex ayuda a leer, diagnosticar y proponer. El kernel ejecuta. Tú revisas el cambio, interpretas el resultado y decides.

2. Preparación mínima, una sola vez

2.1 Instala o confirma estas aplicaciones

- 1 Visual Studio Code.
- 2 Python 3.11 o posterior. Una versión reciente y estable es suficiente.
- 3 Una cuenta de ChatGPT compatible con Codex.

Para comprobar Python en macOS, abre Terminal y ejecuta:

```
python3 --version
```

Si el comando no existe, instala Python antes de continuar. No uses el Python interno de macOS como entorno del curso.

2.2 Instala tres extensiones en VS Code

Abre la vista Extensions con `Cmd+Shift+X` en macOS o `Ctrl+Shift+X` en Windows/Linux. Busca e instala:

- 1 Python, publicada por Microsoft (`ms-python.python`).
- 2 Jupyter, publicada por Microsoft (`ms-toolsai.jupyter`).
- 3 Codex - OpenAI's coding agent, publicada por OpenAI (`openai.chatgpt`).

Comprueba siempre el nombre del publicador. Evita extensiones comunitarias que usan “ChatGPT” o “Codex” en el nombre pero no son la extensión oficial.

2.3 Abre Codex y autentícate

- 1 Selecciona el icono de Codex en la barra lateral de VS Code.
- 2 Si no aparece, abre la Command Palette con `Cmd+Shift+P` y ejecuta `Codex: Open Codex Sidebar`.
- 3 Inicia sesión con tu cuenta de ChatGPT.
- 4 Para este demo, trabaja en modo Local, dentro de la carpeta abierta.

Codex puede usar como contexto los archivos abiertos y las selecciones del editor. También puedes seleccionar código y ejecutar Codex: Add to Thread, o añadir el archivo completo al chat desde la Command Palette.

3. Crea y abre el espacio de trabajo

La carpeta del demo debería tener una estructura parecida a esta:

```
demo-juntos-observaciones/
├── data/
│   ├── estudiantes.csv
│   └── hogares.csv
├── notebooks/
│   ├── 01_demo_inicial.ipynb
│   └── 01_demo_resuelto.ipynb
├── expected/
│   └── resultados_esperados.md
├── requirements.txt
└── README.md
```

Abre la carpeta, no solamente el notebook:

- 1 En VS Code, selecciona File > Open Folder.
- 2 Elige demo-juntos-observaciones.
- 3 Si VS Code pregunta por Workspace Trust, confía únicamente porque la carpeta fue creada y revisada por ti.

Abrir la carpeta completa es importante: Codex puede ver la relación entre el notebook, los datos y los resultados esperados, y las rutas relativas funcionan de manera estable.

4. Crea un entorno de Python aislado

Esta es la ruta más estable para el demo. Abre la terminal integrada con Terminal > New Terminal.

macOS o Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install pandas matplotlib ipykernel
```

Windows PowerShell

```
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install pandas matplotlib ipykernel
```

La instalación mínima usa:

- pandas para abrir, unir y resumir datos;
- matplotlib para cualquier gráfico breve;
- ipykernel para ejecutar Python como kernel del notebook.

No es necesario instalar el paquete completo jupyter para que VS Code ejecute un notebook Python local. Si el repositorio tiene requirements.txt, usa en cambio:

```
python -m pip install -r requirements.txt
```

5. Selecciona intérprete y kernel

El intérprete de Python y el kernel del notebook están relacionados, pero VS Code puede mostrarlos por rutas distintas. Verifica ambos.

5.1 Selecciona el intérprete del proyecto

- 1 Abre la Command Palette.
- 2 Ejecuta Python: Select Interpreter.
- 3 Elige el Python ubicado dentro de `.venv`.

En macOS normalmente termina en:

```
.venv/bin/python
```

En Windows normalmente termina en:

```
.venv\Scripts\python.exe
```

5.2 Selecciona el kernel del notebook

- 1 Abre `notebooks/01_demo_inicial.ipynb`.
- 2 En la esquina superior derecha, selecciona Select Kernel.
- 3 Si aparece `.venv` entre los recientes, selecciónalo.
- 4 Si no aparece, elige Select Another Kernel > Python Environments y selecciona `.venv`.

VS Code recuerda normalmente el último kernel usado por el notebook.

5.3 Ejecuta una prueba de identidad

En una celda nueva, ejecuta:

```
import sys
from pathlib import Path

print("Python:", sys.executable)
print("Carpeta de trabajo:", Path.cwd())
```

La primera línea debe apuntar a `.venv`. La carpeta de trabajo debe permitir que las rutas relativas del notebook encuentren `data/`.

Si el notebook está dentro de `notebooks/`, conviene construir rutas a partir de la raíz del proyecto, no depender de cambiar directorios silenciosamente.

6. Operaciones básicas del notebook

Ejecutar celdas

- Botón triangular a la izquierda: ejecuta una celda.
- `Ctrl+Enter`: ejecuta la celda seleccionada.
- `Shift+Enter`: ejecuta la celda y avanza a la siguiente.
- `Run All`: ejecuta todas las celdas en orden.

En macOS, los atajos de ejecución de celdas también usan `Ctrl`, no `Cmd`.

Celdas de código y Markdown

Usa celdas Markdown para:

- declarar la pregunta;
- explicar la población analítica;

- anticipar qué debería ocurrir;
- interpretar resultados;
- registrar decisiones.

Usa celdas de código para una sola operación conceptual por celda. Para el demo, evita celdas largas que cargan, limpian, unen y resumen todo de una vez.

Reiniciar el kernel

El kernel conserva variables entre ejecuciones. Por eso un notebook puede funcionar después de ejecutar celdas fuera de orden y fallar al abrirlo desde cero.

Antes de la clase, usa la barra superior del notebook para:

- 1 Restart Kernel.
- 2 Run All.
- 3 Confirmar que termina sin depender de memoria previa.

Si una celda queda ejecutándose indefinidamente, usa Interrupt Kernel. Si el estado queda confuso, reinicia el kernel.

Variables y datos

Después de ejecutar celdas, abre Variables desde la barra del notebook. Puedes inspeccionar data frames y abrirlos en el Data Viewer. Esto es útil para verificar columnas, tipos y tamaño sin llenar la presentación de impresiones extensas.

7. Configuración visual para enseñar

Antes del demo:

- 1 Mueve el panel de Codex a la derecha.
- 2 Deja Explorer a la izquierda.
- 3 Usa zoom o una fuente suficientemente grande para proyección.
- 4 Cierra paneles no utilizados, notificaciones y archivos irrelevantes.
- 5 Deja visible el nombre del kernel.
- 6 Colapsa celdas auxiliares que no forman parte del relato.
- 7 Mantén la terminal cerrada hasta necesitarla.

Una composición recomendable es:

Explorer	Notebook	Codex
data/ notebooks/ expected/	Pregunta Código Resultado	Prompt Diagnóstico Cambio propuesto

No intentes mostrar simultáneamente notebook, terminal, Explorer y Codex con el mismo peso. El notebook es el centro; Codex aparece cuando dialogas con él; la terminal aparece solo para verificar o instalar.

8. Flujo seguro con Codex

Paso 1: pedir explicación, sin editar

Abre el notebook y envía:

Lee este notebook sin modificar archivos. Resume la pregunta empírica, la población analítica y cada transformación que pueda cambiar el número o la composición de las observaciones. Señala incertidumbres.

Paso 2: pedir diagnósticos antes de una corrección

No corrijas todavía el notebook. Propón celdas diagnósticas para localizar dónde se pierden observaciones. Incluye conteos antes y después, un indicador de merge y tasas de no correspondencia por Juntos, ruralidad y distrito.

Paso 3: revisar y ejecutar diagnósticos

Puedes copiar las celdas propuestas o pedir a Codex que las inserte. Si edita el notebook:

- 1 revisa el resumen;
- 2 inspecciona el diff;
- 3 acepta solamente las celdas que comprendes;
- 4 ejecuta tú las celdas;
- 5 interpreta el resultado antes del siguiente prompt.

Paso 4: pedir una corrección acotada

Ya confirmamos que el inner merge elimina observaciones de manera desigual. Propón el cambio mínimo que conserve todos los estudiantes durante la auditoría. No alteres todavía la definición final de la muestra. Añade comentarios breves.

Paso 5: convertir el diagnóstico en una protección permanente

Añade dos comprobaciones explícitas: una debe detectar cambios no autorizados en el número de estudiantes; la otra debe fallar si la tasa de correspondencia del merge baja del umbral definido. Explica por qué elegiste cada comprobación.

Paso 6: solicitar una revisión científica, no solo técnica

Compara la tabla inicial y la tabla auditada. Separa claramente:

- 1) lo que estos descriptivos muestran;
- 2) explicaciones alternativas;
- 3) lo que no puede interpretarse causalmente.

No redactes una conclusión causal.

9. Runbook del demo, minuto a minuto

Antes de iniciar

- Abre la carpeta completa.
- Activa `.venv`.
- Selecciona el kernel correcto.
- Reinicia y ejecuta todo el notebook resuelto una vez.
- Vuelve al notebook inicial.
- Abre Codex en un chat nuevo.
- Cierra la terminal.
- Ten abierta una captura o PDF de respaldo.

Minuto 0:00-1:30 - presentar el resultado

- 1 Muestra la pregunta descriptiva.
- 2 Ejecuta las celdas que cargan datos y producen la tabla inicial.
- 3 Pregunta: “¿Qué comprobarían antes de aceptar esta comparación?”

Minuto 1:30-4:00 - comprender antes de editar

- 1 Pide a Codex que explique el notebook.
- 2 Señala que el merge puede cambiar la población.
- 3 No aceptes todavía una corrección.

Minuto 4:00-7:00 - localizar la pérdida

- 1 Añade o ejecuta los conteos antes y después.
- 2 Examina el indicador de merge.
- 3 Compara la pérdida por grupo.
- 4 Verbaliza: “El código corre, pero cambió quién está en los datos.”

Minuto 7:00-9:30 - corregir y proteger

- 1 Pide el cambio mínimo.
- 2 Revisa el diff.
- 3 Ejecuta la corrección.
- 4 Ejecuta las aserciones.

Minuto 9:30-12:00 - interpretar

- 1 Compara resultados.
- 2 Explica por qué una corrección técnica no identifica causalidad.
- 3 Cierra con: “Codex propuso; el kernel ejecutó; nosotros decidimos qué significa.”

10. Qué debe estar preparado en las láminas

No uses las capturas como tutorial de botones. Prepara seis estados:

- 1 La pregunta y los dos archivos.
- 2 La tabla inicial aparentemente convincente.
- 3 La línea del `inner merge` resaltada.
- 4 El conteo antes/después y la composición de los registros perdidos.
- 5 El diff de la corrección y las dos comprobaciones.
- 6 La tabla auditada y el límite inferencial.

Estas imágenes son también el fallback si internet, Codex o el kernel fallan durante la clase.

11. Problemas frecuentes

“Select Kernel” no muestra `.venv`

- 1 Confirma que `.venv` existe dentro de la carpeta abierta.
- 2 Ejecuta en la terminal:

```
.venv/bin/python -m pip install ipykernel
```

En Windows usa `.venv\Scripts\python.exe`.

- 1 Ejecuta Developer: Reload Window.
- 2 Vuelve a Select Kernel > Select Another Kernel > Python Environments.

ModuleNotFoundError

El paquete fue instalado en otro Python. Ejecuta la celda con `sys.executable`, abre una terminal y usa exactamente ese intérprete:

```
python -m pip install NOMBRE_DEL_PAQUETE
```

Evita usar `pip` solo; `python -m pip` reduce el riesgo de instalar en otro entorno.

El notebook funciona solo cuando ejecutas celdas manualmente

Hay estado oculto. Reinicia el kernel y ejecuta Run All. Reordena celdas hasta que el notebook funcione de arriba hacia abajo.

Las rutas a `data/` fallan

Confirma `Path.cwd()`. Abre la carpeta raíz del proyecto y usa rutas construidas desde una raíz explícita. No dependas de cambios manuales de directorio durante el demo.

Codex no usa el notebook correcto

- 1 Mantén abierto el notebook.
- 2 Selecciona las celdas relevantes y usa Add to Thread.
- 3 Añade también `README.md` o `resultados_esperados.md` cuando importen.
- 4 Nombra los archivos en el prompt.

El kernel queda ocupado

Usa Interrupt Kernel. Si no responde, usa Restart Kernel y ejecuta de nuevo desde arriba.

VS Code abre la carpeta en Restricted Mode

No ejecutes código de una carpeta desconocida. Para tu carpeta revisada, abre el indicador de Workspace Trust y marca el espacio como confiable.

Codex propone demasiados cambios

Cancela o rechaza el diff y reduce el alcance del prompt: "Añade solo una celda diagnóstica; no refactorices ni cambios otras celdas."

12. Ensayo y fallback

Un día antes

- Actualiza VS Code y las tres extensiones, y después congela el entorno.
- Reinstala dependencias desde `requirements.txt` en una `.venv` limpia.
- Ejecuta Restart Kernel + Run All.
- Revisa que el notebook inicial falle pedagógicamente de la manera esperada.
- Revisa que el notebook resuelto termine correctamente.
- Graba una ejecución corta o toma las seis capturas.

- Guarda una copia local; no dependas de sincronización en la nube.

Quince minutos antes

- Desactiva notificaciones.
- Conecta el cargador.
- Comprueba proyección y tamaño de fuente.
- Abre la carpeta, el notebook, Codex y la presentación.
- Ejecuta una celda sencilla para comprobar el kernel.
- Envía un prompt corto a Codex para comprobar la sesión.
- Vuelve al estado inicial del demo.

Si Codex falla

Continúa con las respuestas preparadas y pregunta al grupo qué diagnóstico pediría. Ejecuta las celdas ya incluidas en el notebook resuelto.

Si el kernel falla

Usa las capturas de resultados y explica el razonamiento. No gastes tiempo de clase reinstalando Python.

Si el tiempo se reduce

Muestra el diagnóstico ya preparado, ejecuta solo una aserción y conserva el cierre inferencial.

13. Lista corta para imprimir

Preparación

- ☐ VS Code actualizado.
- ☐ Extensiones oficiales Python, Jupyter y Codex activas.
- ☐ Carpeta completa abierta.
- ☐ .venv creada y dependencias instaladas.
- ☐ Intérprete y kernel apuntan a .venv.
- ☐ Notebook inicial y resuelto probados desde kernel limpio.
- ☐ Codex autenticado y funcionando en modo local.
- ☐ Capturas o video de respaldo disponibles.

Durante el demo

- ☐ Explicar antes de editar.
 - ☐ Diagnosticar antes de corregir.
 - ☐ Contar observaciones antes y después.
 - ☐ Revisar quién desaparece.
 - ☐ Inspeccionar el diff.
 - ☐ Ejecutar la corrección.
 - ☐ Añadir comprobaciones permanentes.
 - ☐ Separar resultado descriptivo de inferencia causal.
-

Fuentes oficiales

- Visual Studio Code, Jupyter Notebooks in VS Code:
<https://code.visualstudio.com/docs/datascience/jupyter-notebooks>
- Visual Studio Code, Manage Jupyter Kernels:
<https://code.visualstudio.com/docs/datascience/jupyter-kernel-management>
- Visual Studio Code, Python environments: <https://code.visualstudio.com/docs/python/environments>
- Microsoft Marketplace, Jupyter extension: <https://marketplace.visualstudio.com/items?itemName=ms-toolsai.jupyter>
- OpenAI, Codex IDE extension: <https://developers.openai.com/codex/ide>
- OpenAI Marketplace listing, Codex - OpenAI's coding agent:
<https://marketplace.visualstudio.com/items?itemName=openai.chatgpt>

Las etiquetas exactas de algunos botones pueden variar ligeramente según la versión de VS Code o el sistema operativo. Las rutas principales verificadas son Command Palette, Python: Select Interpreter, Select Kernel y Codex: Open Codex Sidebar.